HybridRAG: Implementation Report

Integrating Knowledge Graphs and Vector Retrieval for
Enhanced Information Extraction from Financial Documents

Extended with Custom Ontology-Based Knowledge Graph Construction

Bibek Singh Dhody

February 2026

Abstract

This report presents the implementation of HybridRAG, a Retrieval-Augmented Generation system that combines Knowledge Graphs with Vector-based retrieval for enhanced question answering over financial documents. We describe the complete system architecture, implementation details, and present an extension that enables custom ontology-based knowledge graph construction using Large Language Models (LLMs). The extension allows domain experts to define their own entity types and relationship schemas, which are then used by an LLM (Ollama) to intelligently extract structured knowledge from unstructured text. This approach significantly improves domain-specific information extraction compared to traditional Named Entity Recognition (NER) methods.

Contents

1 Introduction

1.1 Background and Motivation

Financial documents such as SEC filings, earnings reports, and annual reports contain rich structured and unstructured information. Traditional Retrieval-Augmented Generation (RAG) systems rely solely on vector similarity search, which often fails to capture the complex relationships between entities mentioned in these documents.

HybridRAG addresses this limitation by combining:

- **Vector RAG**: Semantic similarity-based retrieval using dense embeddings
- **Graph RAG**: Structured knowledge retrieval from Knowledge Graphs
- **Hybrid Fusion**: Intelligent aggregation of both retrieval methods

1.2 Problem Statement

The key challenges addressed by this implementation include:

1. Extracting structured knowledge from unstructured financial documents
2. Building domain-specific knowledge graphs with custom entity and relationship types
3. Combining structured (graph) and unstructured (vector) retrieval methods
4. Enabling domain experts to define custom ontologies without programming expertise

1.3 Contributions

This implementation makes the following contributions:

1. A complete Java-based HybridRAG system with Vector RAG, Graph RAG, and Hybrid fusion
2. Integration with multiple LLM providers (Ollama, OpenAI, Gemini)
3. **Extension**: A novel LLM-based knowledge extraction pipeline that uses custom ontologies to intelligently extract domain-specific entities and relationships

2 System Architecture

2.1 Overall Architecture

The HybridRAG system consists of several interconnected components as shown in Figure 1.

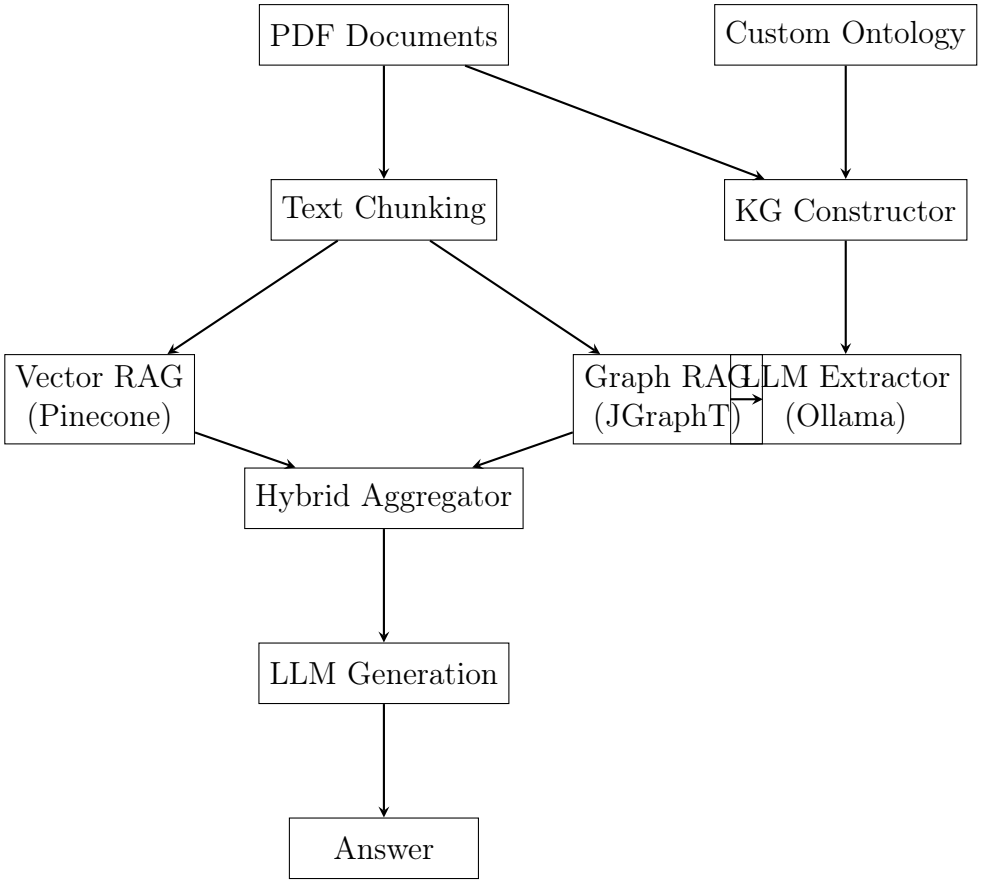

Figure 1: HybridRAG System Architecture

2.2 Component Overview

2.2.1 Vector RAG Component

The Vector RAG component handles semantic similarity-based retrieval:

- **Embedding Generation**: Uses Gemini API for generating 768-dimensional embeddings
- **Vector Storage**: Pinecone vector database for efficient similarity search
- **Retrieval**: Top-K retrieval based on cosine similarity

2.2.2 Graph RAG Component

The Graph RAG component handles structured knowledge retrieval:

- **Graph Storage**: JGraphT library for in-memory graph operations
- **Entity Matching**: Fuzzy matching to find relevant entities in queries
- **Subgraph Extraction**: DFS-based traversal to extract relevant triplets

2.2.3 Knowledge Graph Constructor

The KG Constructor supports two extraction modes:

- **NER Mode**: Stanford CoreNLP-based Named Entity Recognition
- **LLM Mode**: Custom ontology-driven extraction using Ollama

3 Implementation Details

3.1 Technology Stack

Table 1: Technology Stack

Component	Technology	Purpose
Programming Language	Java 17	Core implementation
Build Tool	Maven	Dependency management
PDF Processing	Apache PDFBox	Text extraction from PDFs
NLP	Stanford CoreNLP	Named Entity Recognition
Graph Library	JGraphT	Knowledge Graph operations
Vector Database	Pinecone	Embedding storage and retrieval
LLM (Local)	Ollama (llama3.1:8b)	Text generation and extraction
LLM (Cloud)	Gemini / OpenAI	Embeddings and generation

3.2 Project Structure

```
src/main/java/com/hybridrag/
          Main.java                      # CLI entry point
          config/
                HybridRAGConfig.java     # Configuration management
          model/
                CustomOntology.java      # Ontology schema definition
                Document.java            # Document representation
                DocumentChunk.java       # Text chunk model
                Entity.java              # Entity model
                Relationship.java        # Relationship model
                RAGResult.java           # Query result model
          service/
              VectorRAG.java             # Vector retrieval
              GraphRAG.java              # Graph retrieval
              HybridRAG.java             # Hybrid aggregation
              KnowledgeGraphConstructor.java  # KG building
              LLMKnowledgeExtractor.java      # LLM-based extraction
              OllamaService.java         # Ollama API client
              GeminiService.java         # Gemini API client
              PineconeRestClient.java    # Pinecone API client
```

Listing 1: Project Directory Structure

3.3 Core Data Models

3.3.1 Entity Model

```java
public class Entity {
    private String id;
    private String name;
    private String type;  // e.g., COMPANY, PERSON, PRODUCT
    private Map<String, Object> properties;
}
```

Listing 2: Entity Class

3.3.2 Relationship Model

```java
public class Relationship {
    private String id;
    private String sourceId;
    private String targetId;
    private String type;  // e.g., PRODUCES, LED_BY, LOCATED_IN
    private Map<String, Object> properties;
}
```

Listing 3: Relationship Class

4 Extension: Custom Ontology-Based Knowledge Graph Construction

This section describes the major extension to the original HybridRAG system: the ability to define custom domain ontologies and use LLM-based extraction to build knowledge graphs.

4.1 Motivation

Traditional NER-based knowledge extraction has significant limitations:

- **Fixed Entity Types**: Standard NER models recognize only predefined types (PERSON, ORGANIZATION, LOCATION, etc.)
- **No Domain Specificity**: Cannot extract domain-specific entities like FINANCIAL_METRIC, DRUG, or LEGAL_CONCEPT
- **Limited Relationships**: Dependency parsing captures syntactic relationships, not semantic ones
- **Keyword Matching is Insufficient**: Simple keyword matching cannot understand context or synonyms

4.2 Solution: LLM-Based Ontology-Driven Extraction

Our extension uses Large Language Models to intelligently extract entities and relationships based on a user-defined ontology schema. The key insight is:

> *The ontology defines WHAT to look for (schema), NOT explicit keywords. The LLM does the actual extraction by UNDERSTANDING the text semantically.*

4.3 Custom Ontology Schema

The ontology is defined as a JSON file with the following structure:

```
{
  "name": "Financial Document Ontology",
  "description": "Ontology for financial documents",
  "domain": "finance",
  "entityTypes": [
    {
      "type": "COMPANY",
      "description": "A business organization or corporation",
      "examples": ["Apple Inc.", "Microsoft", "Tesla"]
    },
    {
      "type": "FINANCIAL_METRIC",
      "description": "A quantifiable financial measure",
      "examples": ["revenue", "net income", "EPS"]
    }
  ],
  "relationTypes": [
    {
      "type": "REPORTED",
      "description": "Company reported a financial metric",
      "sourceType": "COMPANY",
      "targetType": "FINANCIAL_METRIC"
    }
  ]
}
```

Listing 4: Custom Ontology Structure

4.4 LLM-Based Extraction Pipeline

Algorithm 1 LLM-Based Knowledge Extraction

Require: Document D, Ontology O
Ensure: Knowledge Graph $G = (E, R)$

$E \leftarrow \emptyset$ ▷ Entity set
$R \leftarrow \emptyset$ ▷ Relationship set
$chunks \leftarrow \text{ChunkDocument}(D, 1500)$ ▷ Split into chunks
for each $chunk$ in $chunks$ **do**
    $prompt \leftarrow \text{BuildExtractionPrompt}(chunk, O)$
    $response \leftarrow \text{CallOllama}(prompt)$
    $(e, r) \leftarrow \text{ParseJSON}(response)$
    $e' \leftarrow \text{ResolveEntities}(e, E)$ ▷ Entity resolution
    $r' \leftarrow \text{ValidateRelationships}(r, O)$ ▷ Validation
    $E \leftarrow E \cup e'$
    $R \leftarrow R \cup r'$
end for
return (E, R)

4.5 Extraction Prompt Design

The extraction prompt is carefully designed to guide the LLM:

```
You are a knowledge extraction system. Extract entities and
relationships from the text according to the ontology.

## Domain: finance

## Entity Types to Extract:
- **COMPANY**: A business organization...
- **FINANCIAL_METRIC**: A quantifiable financial measure...
- **MONEY**: A specific monetary value...

## Relationship Types to Extract:
- **REPORTED** [COMPANY -> FINANCIAL_METRIC]: Company reported...
- **HAS_VALUE** [FINANCIAL_METRIC -> MONEY]: Metric has value...

## Text to analyze:
{chunk_text}

## Output Format:
Return ONLY valid JSON:
{
  "entities": [{"name": "...", "type": "..."}],
  "relationships": [{"source": "...", "target": "...",
                     "type": "..."}]
}
```

Listing 5: LLM Extraction Prompt Template

4.6 Entity Resolution

Entity resolution merges multiple mentions of the same real-world entity:

```java
private Map<String, Entity> resolveEntities(List<Entity> entities) {
    Map<String, Entity> resolved = new HashMap<>();
    for (Entity entity : entities) {
        String canonical = canonicalize(entity.getName());
        if (resolved.containsKey(canonical)) {
            // Merge with existing entity
            Entity existing = resolved.get(canonical);
            existing.addAlias(entity.getName());
        } else {
            resolved.put(canonical, entity);
        }
    }
    return resolved;
}

private String canonicalize(String name) {
    // Remove suffixes like "Inc.", "Corp.", "Ltd."
    // Normalize whitespace and case
    return name.replaceAll("\\s+(Inc|Corp|Ltd)\\.?$", "")
               .trim().toLowerCase();
}
```

Listing 6: Entity Resolution Algorithm

4.7 Relationship Validation

Relationships are validated against the ontology constraints:

```java
public boolean isValidRelationship(String relationType,
                                   String sourceType,
                                   String targetType) {
    for (RelationType rt : relationTypes) {
        if (rt.getType().equals(relationType)) {
            // Check source type constraint
            if (rt.getSourceType() != null &&
                !rt.getSourceType().equals(sourceType)) {
                return false;
            }
            // Check target type constraint
            if (rt.getTargetType() != null &&
                !rt.getTargetType().equals(targetType)) {
                return false;
            }
            return true;
        }
    }
    return false;
}
```

Listing 7: Relationship Validation

5 Supported Domains

The system provides pre-built ontologies for several domains:

5.1 Financial Domain

Table 2: Financial Domain Ontology

Entity Types	**Relationship Types**
COMPANY	REPORTED (Company → Metric)
PERSON	LED_BY (Company → Person)
FINANCIAL_METRIC	PRODUCES (Company → Product)
MONEY	HAS_VALUE (Metric → Money)
PERCENTAGE	LOCATED_IN (Company → Location)
PRODUCT	DURING_PERIOD (* → Date)
DATE	INCREASED / DECREASED
LOCATION	ACQUIRED (Company → Company)

5.2 Medical Domain

Table 3: Medical Domain Ontology

Entity Types	**Relationship Types**
DRUG	TREATS (Drug → Disease)
DISEASE	CAUSES (* → *)
GENE	HAS_SYMPTOM (Disease → Symptom)
SYMPTOM	INHIBITS (Drug → Gene)
TREATMENT	ADMINISTERED_AT (Drug → Dosage)
DOSAGE	

5.3 Legal Domain

Table 4: Legal Domain Ontology

Entity Types	**Relationship Types**
PARTY	SUED_BY (Party → Party)
COURT	PRESIDED_BY (* → Judge)
JUDGE	FILED_IN (* → Court)
CASE_NUMBER	CITED_STATUTE (* → Statute)
STATUTE	RULED_THAT (* → Legal Concept)
LEGAL_CONCEPT	AWARDED (Party → Money)
DATE, MONEY	OCCURRED_ON (* → Date)

6 Experimental Results

6.1 Dataset

We evaluated the system using Apple Inc.'s Q1 2024 10-K filing (80 pages, 273,314 characters).

6.2 Knowledge Graph Statistics

Table 5: Extraction Results from Apple 10-K Document

Metric	Value
Document Pages	80
Total Characters	273,314
Text Chunks Processed	243
Entities Extracted	**987**
MONEY	246
FINANCIAL_METRIC	184
PRODUCT	166
COMPANY	123
DATE	104
LOCATION	74
PERSON	63
PERCENTAGE	27
Relationships Extracted	**518**
PRODUCES	150
LOCATED_IN	106
REPORTED	91
DURING_PERIOD	86
LED_BY	29
DECREASED	21
INCREASED	16
HAS_VALUE	15
ACQUIRED	4

6.3 Sample Query Results

6.3.1 Query 1: Products

```
Question: What products does Apple Inc produce?

Answer: Apple Inc. produces the following products:
1. iPhone              13. Mac mini
2. macOS               14. iMac
3. iOS                 15. iPad
4. App Store           16. MacBook Air
5. Apple Arcade        17. Mac Studio
6. Apple Fitness+      18. watchOS 26
7. Apple Music         19. visionOS 26
8. Apple News+         20. tvOS 26
9. Apple TV 4K         21. Apple Watch Series 11
10. Apple Card         22. Apple Watch SE 3
11. Apple Pay          23. Apple Watch Ultra 3
12. MacBook Pro        24. AirPods
```

6.3.2 Query 2: Leadership and Location

```
Question: Who leads Apple Inc and where is it located?

Answer: Apple Inc. is led by Tim Cook (Timothy D. Cook).
Locations: Cupertino, California, United States, and
international operations in China, India, Japan,
South Korea, Taiwan, Vietnam, Europe (Ireland).
```

7 Usage Guide

7.1 Generating a Custom Ontology

```
# Generate financial ontology (default)
mvn exec:java -Dexec.mainClass="com.hybridrag.Main" \
  -Dexec.args="--mode generate-ontology"

# Generate medical ontology
mvn exec:java -Dexec.mainClass="com.hybridrag.Main" \
  -Dexec.args="--mode generate-ontology --domain medical"

# Generate legal ontology
mvn exec:java -Dexec.mainClass="com.hybridrag.Main" \
  -Dexec.args="--mode generate-ontology --domain legal"
```

Listing 8: Generate Sample Ontology

7.2 Building Knowledge Graph with Custom Ontology

```
# Ensure Ollama is running
ollama run llama3.1:8b

# Build KG with custom ontology
mvn exec:java -Dexec.mainClass="com.hybridrag.Main" \
  -Dexec.args="--mode build-kg-custom \
               --documents data/documents/file.pdf \
               --ontology data/custom_ontology.json"
```

Listing 9: Build Knowledge Graph

7.3 Querying the Knowledge Graph

```
# Query using Graph RAG only
mvn exec:java -Dexec.mainClass="com.hybridrag.Main" \
  -Dexec.args="--mode query-kg \
               --question 'What products does Apple produce?'"

# Query using Hybrid RAG (requires Pinecone)
mvn exec:java -Dexec.mainClass="com.hybridrag.Main" \
  -Dexec.args="--mode query \
               --question 'What was Apple revenue?'"
```

Listing 10: Query Knowledge Graph

8 Comparison: NER vs LLM-Based Extraction

Table 6: Comparison of Extraction Methods

Aspect	**NER-Based**	**LLM-Based (Ours)**
Entity Types	Fixed (7-10 types)	Unlimited (user-defined)
Domain Adaptation	Requires retraining	Schema change only
Relationship Extraction	Syntactic patterns	Semantic understanding
Context Understanding	Limited	Full contextual
Processing Speed	Fast	Slower (API calls)
Setup Complexity	High (CoreNLP)	Low (Ollama)
Customization	Difficult	JSON schema

9 Conclusion and Future Work

9.1 Conclusion

This implementation report presented HybridRAG, a system that combines Knowledge Graphs with Vector Retrieval for enhanced question answering over financial documents. The key contribution is the extension that enables custom ontology-based knowledge graph construction using LLMs.

The LLM-based approach offers several advantages:

- **Flexibility**: Domain experts can define custom entity and relationship types
- **Semantic Understanding**: LLMs understand context and synonyms
- **No Retraining**: New domains require only a schema change
- **Structured Output**: Validated against ontology constraints

9.2 Future Work

1. **Incremental Updates**: Support for adding new documents without rebuilding the entire graph
2. **Graph Embeddings**: Learn entity embeddings for better similarity search
3. **Multi-hop Reasoning**: Support complex queries requiring multiple inference steps
4. **Evaluation Framework**: Comprehensive metrics for extraction quality
5. **UI Interface**: Web-based interface for ontology design and querying

Acknowledgments

This work builds upon the HybridRAG research paper and extends it with custom ontology-based knowledge extraction capabilities.

References

[1] HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction.

[2] Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.

[3] Hogan, A., et al. (2021). Knowledge Graphs. *ACM Computing Surveys*.

[4] Ollama: Run Large Language Models Locally. `https://ollama.ai`

[5] Pinecone: Vector Database for Machine Learning. `https://pinecone.io`